RETRACE: MODEL-RESPONSE WATERMARKING FOR DIFFUSION LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Existing language-model watermarks are detected from token patterns. We detect ours from *how a diffusion language model responds when the generated text is partially masked and reconstructed*. Because such a model is trained to recover masked tokens from arbitrary surviving context, a finished text can be pushed back through the operator that produced it: corrupted with a secret key, re-denoised, and its response read. ReTrace places a watermark in that response. A keyed pair of context ablations induces a per-position log-likelihood contrast, and the watermark asks the sign of that contrast to agree with a keyed orientation, giving a test statistic that is a sum of bounded indicators with a finite-sample exact Binomial$(n, \frac{1}{2})$ null. Embedding never overrides the model: decoding follows the reference block schedule, and the watermark only decides whether a model-preferred proposal is committed now or deferred. We also report the negative result that motivated this design — post-hoc token substitution is priced out on a well-trained diffusion language model, and, because both are governed by the sharpness of the same conditionals, *improving generation quality reduces post-hoc watermarkability*.

1 INTRODUCTION

Existing language-model watermarks encode provenance in *what tokens are produced*. Red–green schemes bias the token distribution toward a keyed vocabulary partition (Kirchenbauer et al., 2023); distortion-free schemes fix the sampling randomness with a keyed pseudorandom sequence; and the watermarks designed for diffusion language models exploit the extra freedom those models expose — unresolved context, per-step sampling randomness, global sequence statistics, or the order in which positions are unmasked (Hong & No, 2026). Different as they are, every one of them leaves the verifier reading a *surface signature*: a hash of token identities, a frequency, a rank. We ask a different question.

Can a watermark be detected from how the model reconstructs a text, rather than from the tokens the text contains?

A diffusion language model makes this question answerable in a way an autoregressive model does not. Its training objective is to recover masked tokens from arbitrary surviving context, so any finished text can be pushed *back* through the operator that produced it: corrupt it, re-denoise it, and observe the response. That response is a property of the text's relationship to the model, not of its symbols, and it is available to anyone holding the model — but only interpretable by someone holding the key that chose the corruption.

We call the resulting scheme **ReTrace**. A secret key derives two corruption patterns over the context of a span. Reading the model's log-probability of each *observed* token under both corruptions gives a contrast

$$g_i(y_i) \;=\; \log p_\theta\big(y_i \mid C_i^1(y)\big) - \log p_\theta\big(y_i \mid C_i^0(y)\big)\,, \tag{1}$$

and the watermark asks not that y_i belong to a keyed vocabulary subset, but that the sign of $g_i(y_i)$ agree with a keyed orientation. The evidence lives in the model's behaviour.

Getting the embedding right took one full negative result, which we report because it is the reason the final design looks the way it does. The obvious way to place such a watermark is to *substitute* tokens: at keyed positions, replace the model's choice with an admissible alternative whose contrast

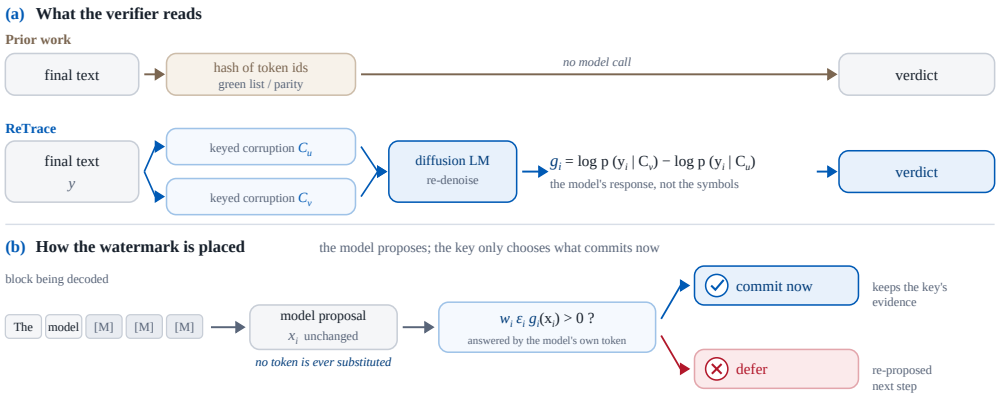

Figure 1: ReTrace embeds through the denoising trajectory but verifies through the model's response to a keyed reconstruction. **(a)** Prior watermarks hand the verifier a function of the token identities — a green-list membership, a parity, a hash. ReTrace hands it the model: the finished text is corrupted twice under the key and re-denoised, and the contrast between the two reconstructions of the *observed* token is the evidence. The signal is therefore a property of the text's relationship to the model rather than of its symbols, and it is recoverable from the text, the model and the key alone, with no access to the generation trajectory. **(b)** Placing it costs almost nothing because nothing is overridden. Decoding follows the reference block schedule and the model proposes what it would have proposed; the key decides only whether a proposal is committed now or deferred to a later step, when more of the block has resolved.

points the right way. On a well-trained diffusion language model this does not work, and the reason is a property of the model rather than of the algorithm. Under reference decoding the denoiser's conditionals are sharp enough that the median position admits exactly one token inside any usable fluency budget, so every unit of watermark evidence must be bought with a unit of fluency. Worse, the effect strengthens as generation improves: repairing a sampler bug in our own harness lowered draft perplexity from 22.2 to 9.5 and simultaneously halved the denoiser's entropy and shrank the admissible set by 34–60%. We state this as a finding in its own right — *better diffusion-model generation reduces post-hoc watermarkability* — and it rules out an entire family of designs.

What survives is to stop changing *which* token is produced and change only *when* a model-preferred token is committed. Decoding a block, the model proposes what it would have proposed anyway; a position whose proposal already answers the keyed challenge is committed first, and one that does not is deferred. No token is ever overridden, which is why this costs almost nothing — the same reason decoding-order watermarks are cheap (Hong & No, 2026). The difference is what the verifier reads: order is only the *embedding* channel here, while the evidence recovered at verification time is the model's response to a keyed reconstruction.

ReTrace places the watermark in the decoding order and reads it back from the model's reconstruction response (Figure 1).

Contributions.

- A watermark read from the model's response rather than from token patterns: the response of a finished text to keyed re-denoising, readable from the text, the model and the key alone, with no access to the generation trajectory (§3).
- A test statistic that is a sum of bounded indicators with one keyed orientation bit per position, giving a finite-sample exact Binomial$(n, \frac{1}{2})$ null under the ideal-PRF randomization model, with presence, claimed-message verification and payload decoding kept separate (§3.2). False-positive control is verified per key and at the worst key, not averaged over keys.
- An embedding rule that never substitutes a token, steering only the commit order inside each block of the reference decoding schedule (§3.3).

- A negative result with a mechanism: post-hoc substitution is priced out on modern diffusion language models, and improving generation quality makes it worse (§4).

2 Related Work

Diffusion language models. Continuous diffusion (Ho et al., 2020) was carried to discrete data by structured corruption processes (Austin et al., 2021) and by score-ratio and masked-diffusion formulations (Lou et al., 2024; Sahoo et al., 2024; Shi et al., 2024; Ou et al., 2025). Instruction-following models at language-model scale followed — LLaDA and its successors (Nie et al., 2025; Zhu et al., 2025), Dream (Ye et al., 2025) — alongside industrial systems built on the same paradigm (Labs et al., 2025). The property we exploit is the training objective itself: these models predict masked tokens from arbitrary surviving context, so they can be applied to a *finished* text and not only used to produce one.

Two further findings from this literature shape our design. First, practical dLLMs are *not* order-invariant even though an ideal conditional predictor would be, and the choice of unmasking order measurably changes what is generated (Kim et al., 2025). Second, decoding is in practice semi-autoregressive: blocks are filled in sequence and diffusion happens within a block (Arriola et al., 2025), and work on accelerating that loop shows that committing a confident token whose left context is still unresolved damages coherence (Wu et al., 2026; Wei et al., 2026; Ben-Hamu et al., 2025). Our embedder inherits both: it steers only *within* the reference block schedule, and it borrows the frontier-anchored commit rule rather than inventing a schedule of its own.

Watermarks for autoregressive models. The dominant family biases token probabilities. Kirchenbauer et al. (2023) seed a green/red vocabulary partition on the preceding token and add a logit bias; detection counts green tokens against a binomial null, and the scheme's reliability and robustness have been studied extensively (Kirchenbauer et al., 2024; Zhao et al., 2023; 2024). A second family avoids distortion by fixing the sampling randomness with a keyed pseudorandom sequence, either through the Gumbel-max trick (Aaronson & Kirchner, 2023; Fu et al., 2024) or with explicit distortion-freeness and undetectability guarantees (Kuditipudi et al., 2024; Christ et al., 2024); others place the signature in model parameters rather than in sampling (Block et al., 2025). Surveys cover the space (Petitcolas et al., 1999; Liang et al., 2026), and the threat model is correspondingly well studied, from paraphrase attacks (Krishna et al., 2023) to extraction of the key itself (Jovanović et al., 2024). What every one of these has in common is the *verification* step: the detector computes a function of the token identities.

Watermarks beyond left-to-right generation. Because the green list is seeded by a prefix that order-agnostic decoding does not provide, transferring the autoregressive constructions to dLLMs requires care. Chen et al. (2025) give a watermark for order-agnostic models by biasing token selection in a way that does not assume a prefix. For diffusion language models specifically, Gloaguen et al. (2026) apply a red–green bias *in expectation* over unresolved context and additionally prefer tokens that make other positions green; Bagchi et al. (2025) use keyed Gumbel-max sampling at each denoising step while preserving the sampling distribution; Wu et al. (2025) and Raban et al. (2026) target the same setting with order-agnostic and efficiency-oriented constructions. Closest to our embedding channel is dgMARK (Hong & No, 2026), which leaves the learned conditionals untouched and instead prefers to unmask positions whose already-preferred token satisfies a keyed parity condition; that discipline is precisely why it is inexpensive, and we adopt it.

What is new here. All of the above differ in *where the watermark is placed* — token probabilities, sampling randomness, decoding order — while agreeing on *what the verifier reads*: a function of the emitted symbols. ReTrace changes the second. The verifier masks part of the finished text under the key, has the model reconstruct it, and compares the reconstructions; the evidence is the model's response, which no token-level statistic can express. The price is equally clear and we report it in every table: those detectors need no model calls, ours needs a fixed number of sequence evaluations. To our knowledge no prior watermark for language models — autoregressive or diffusion — uses the generator itself as the verification oracle in this way.

3 RETRACE

3.1 THE OBSERVABLE

Let y be a generated span and K a secret key. For a block B of y, the key derives L *ablation patterns* $D_1, \ldots, D_L$ over the context preceding B, and a pair (u, v) among them. Writing $\mathscr{C}_\ell$ for the context with B, everything after B, and D_ℓ masked, the *challenge contrast* at position $i \in B$ is

$$g_i(w) \;=\; \log p_\theta(w \mid \mathscr{C}_v) - \log p_\theta(w \mid \mathscr{C}_u)\,, \qquad w \in \mathscr{V}. \tag{2}$$

Two properties of $\mathscr{C}_\ell$ matter and neither is optional.

The block and its future are masked in every arm. While B is being decoded, every later block is still `[MASK]`. If the verifier recomputed Eq. (2) on the finished text without re-masking that tail, it would condition on tokens that did not exist when the guidance was produced, and the two sides would be reading different functions. Masking B and everything after it makes the table identical at both times; masking B itself additionally makes it *invariant* to whatever is committed inside B, so a single table serves the block's entire decoding.

The ablations are drawn from the already-final region. $D_\ell \subset \text{prompt} \cup B_{<b}$, which the verifier has verbatim. We use a *contrast* pair — one pattern ablating the context nearest the block, the other the furthest — because recency dominates next-token prediction, so the two arms disagree far more than two random ablations do, widening the dynamic range of g.

3.2 THE STATISTIC

Each position carries an independent keyed orientation bit $\varepsilon_i \in \{\pm 1\}$ and a target $w_i \in \{\pm 1\}$, and contributes a bounded indicator

$$m_i \;=\; \mathbf{1}[w_i\,\varepsilon_i\,g_i(y_i) > 0]\,, \qquad T \;=\; \sum_{i \in P} m_i. \tag{3}$$

Proposition 1 (Exact null)**.** *Let the challenge patterns, the per-position pair selection, the carrier set, the group assignment, the tie coins and the orientation bits each be derived from the key under* separate PRF domains*. Fix a text y independent of the key, and condition on the carrier set, the selected pairs, and the challenge magnitudes — all of which are functions of y and the non-orientation domains only. Under the ideal-PRF model the orientation bits are then independent Rademacher variables independent of everything conditioned on, so the m_i are i.i.d.* Bernoulli$(\frac{1}{2})$ *and* $T \sim$ Binomial$(|P|, \frac{1}{2})$ *exactly.*

The domain separation is load-bearing, not stylistic: the challenge patterns and the per-position pair selection are themselves keyed, so if they shared PRF output with the orientation bits, the conditioning event would carry information about the bits and the independence step would fail. Pair selection additionally uses only $S_i = 2\min(q_i^+, q_i^-)$, which is invariant under swapping the pair's arms, so it is orientation-symmetric by construction.

The proposition needs no calibration corpus and no model-specific null estimate, and it is finite-sample rather than asymptotic. Three implementation details are load-bearing.

Ties must be resolved, not dropped. The model is confident enough that at a large fraction of positions the emitted token's log-probability rounds to 0 under *both* arms in single precision, so their difference vanishes. Scoring $g_i = 0$ as a miss makes $\Pr[m_i = 1] = (1 - \Pr[g_i = 0])/2$; we measured an unwatermarked match rate of 0.32–0.37 against 0.50. We compute Eq. (2) in double precision inside the softmax, which removes the ties entirely, and break any residual tie with a second keyed coin.

Counting, not averaging. An earlier design averaged g over the positions of a probe. The per-position contrast is heavy-tailed, so the mean is dominated by a few large values and controlling the *sign* of the small ones — all a commit-order channel can do — has almost no leverage on it. A count gives every steered position exactly one unit.

Presence, verification and payload are separate tests. Summing hits across payload groups cancels: with a balanced message, half the groups are driven toward $m_i = 1$ and half toward $m_i = 0$,

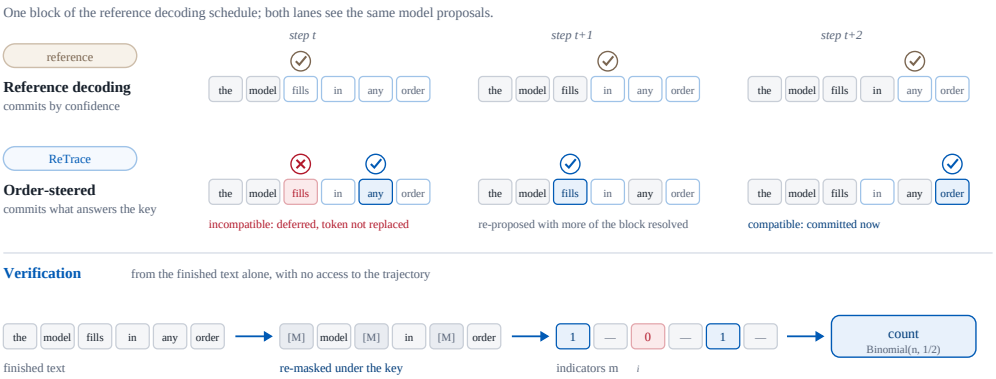

Figure 2: The two lanes see the same model proposals; only the order of commitment differs. Reference decoding commits whichever eligible position it is most confident about. ReTrace commits a position whose *already-preferred* token answers the keyed challenge and defers one whose does not — the deferred position is not overwritten, it is re-proposed at the next step with more of the block resolved, and may then be compatible. Because no token is ever replaced, the price of the watermark is not paid in fluency. The strip below shows why the trajectory need not be recorded: the verifier re-masks the finished text under the key, reads one bounded indicator per position it can score, and counts. Positions the channel cannot steer contribute nothing and are excluded rather than counted as misses, which is what keeps the count's null exact.

so a *perfect* watermark still sums to $|P|/2$. We therefore reserve a presence pool with $w_i \equiv +1$, whose count tests *is this a ReTrace text* with no claimed message, and which also avoids decoding a message from the data and then testing significance against the message just decoded. Verification of a claimed message aligns every group to its claimed bit and uses all carriers; payload decoding thresholds each group separately.

3.3 Embedding by commit order

Generation follows the reference schedule of the base model: blocks in order, diffusion within a block (Figure 2). At each step the model proposes candidates $\hat{x}_i$ with confidences c_i exactly as it would without a watermark. *No proposal is ever replaced.* The watermark decides only which position is committed:

$$w_i \varepsilon_i g_i(\hat{x}_i) > 0 \;\Rightarrow\; \text{commit}, \qquad \text{otherwise} \;\Rightarrow\; \text{defer}. \tag{4}$$

A deferred position is re-proposed at the next step with more of the block resolved, and may then be compatible.

Commit safety follows the frontier-anchored rule of §2: a position is eligible when $c_i \geq \tau$, and only the eligible run surviving at most H still-masked-but-ineligible positions to its left may be committed; a scheduled top-k branch guarantees the block finishes on time.

Only steerable positions are carriers. Committing compatible positions first is *zero-sum* unless deferral changes what the model proposes, because every position is eventually committed and the residual is precisely the incompatible ones. We measured this exactly: the positions the watermark drove matched their target at $107/107$, $133/133$ and $112/112$, while the leftovers matched at 0.06–0.15, summing to 0.50. The statistic must therefore count only positions the channel can move. We define these by the top-2 log-probability gap under the block-masked conditional — a context containing none of the block's own tokens, so the verifier reconstructs the identical carrier set from the finished text — and spend forced commits on non-carrier positions first, so a steerable position keeps its chance to be re-proposed compatibly.

4 WHY POST-HOC SUBSTITUTION FAILS

The first version of ReTrace embedded by substitution: at keyed carrier positions, replace the model's token with an admissible alternative whose contrast points the right way, subject to a per-token fluency budget τ (a substitution is admissible iff $p(w)/p(w^\star) \geq e^{-\tau}$). It does not work, and the reason generalises beyond our implementation.

There is nothing to substitute. Across 36 configurations spanning three carrier-selection rules, three budgets and two guidance weights, the median carrier position admits *exactly one* token at $\tau = 3$, i.e. $p(w)/p(w^\star) \geq 0.05$, with no truncation of the candidate slate. The best configuration inside a $1.35\times$ perplexity budget reaches TPR@1% $= 0.10$; matched local dgMARK reaches 0.86 at $1.23\times$.

Better generation makes it worse. Our harness initially ranked positions for low-confidence re-masking by the maximum of the Gumbel-perturbed logits rather than by $p(x_0)$ under the clean softmax. Repairing it to match the reference sampler moved draft perplexity from 22.2 to 9.5 — and halved the denoiser's entropy, from 0.655 to 0.319, shrinking the admissible set by 34–60 % ($|A(3)|$: $2.5 \to 1.6$; $|A(6)|$: $14.1 \to 5.6$). Part of the early design's apparent strength came from a degraded generator. *Improving a diffusion language model's generation quality reduces its post-hoc watermarkability*, because both quantities are governed by the sharpness of the same conditionals.

Moving to generation time does not rescue substitution. A guidance table valid during generation requires every non-carrier token to be final before it is computed, which forces an unusual schedule: all non-carrier positions, then all carrier positions. Priced with the watermark switched off, that schedule alone costs $1.98\times$ perplexity — more than dgMARK's entire watermark. The lesson is not that generation-time embedding is wrong but that a *global* carrier is: making the carrier block-local reduces the isolated region from half the sequence to one block, and lets the reference schedule stand.

Deferral, by contrast, does change the proposal. The commit-order channel rests on one assumption: that a position which is deferred returns a *different* token later. It does. Measured over full decoding runs, a deferred position's proposal changes 38–51 % of the time, and monotonically in how long it waits — 0.20 after one or two steps, 0.35 after three to five, 0.49 after six to ten, 0.57 after eleven or more. The channel therefore needs step budget rather than a stronger push: a position must wait roughly six steps before a fresh draw is likely, so a schedule that gives a 32-position block only 32 steps finishes before long deferrals can pay.

What this rules in. Substitution buys evidence with fluency at a fixed exchange rate set by the model's conditionals, and that rate is unfavourable and worsening. Any viable embedding must therefore avoid overriding the model's choice — which is what §3.3 does.

5 EXPERIMENTAL SETUP

All numbers are measured on one NVIDIA TITAN RTX with `GSAI-ML/LLaDA-8B-Instruct` in fp16. Prompts are C4 `realnewslike` documents truncated to 300 characters, which is dgMARK's published protocol; every method in this paper uses the same construction, the same checkpoint and 256 generated tokens. Text quality is GPT-2-large perplexity, reported as a ratio against a non-watermarked control *produced under the same decoding regime*, since the three methods require different regimes and a single shared baseline would charge the cost of a regime to whichever watermark used it. Detectability is reported as the true positive rate at a controlled false positive rate, never as a raw z.

Two observations about the baselines. Locally reproduced dgMARK is weaker than its published table ($1.25\times$ for 99.4 %), which is why the reproduction was worth running rather than citing. And KGW's apparent $1.03\times$ is against its own left-to-right control; charged against block decoding it is $1.20\times$, and under forced left-to-right decoding this model repeats about a third of its bigrams, so its detector must score each distinct bigram once — an un-deduplicated detector measured a 37 % false positive rate on non-watermarked text.

Method	Setting	PPL ↓	Ratio ↓	TPR at FPR ↑ 5%	1%	0.1%	z	Det. fwd. ↓
Generation-time watermarks (the watermark is applied while the text is produced)								
No watermark	block dec., top-k 3	9.94	1.00×	–	–	–	+0.04	–
dgMARK (Hong & No, 2026)	$k = 1$	15.40	1.55×	0.88	0.74	0.62	+4.35	**0**
dgMARK (Hong & No, 2026)	+3-beam	12.23	1.23×	**0.92**	**0.86**	**0.82**	+5.81	**0**
No watermark	left-to-right	11.61	1.00×	–	–	–	+0.24	–
KGW (Kirchenbauer et al., 2023)	$\delta = 1$	11.96	**1.03**×	0.97	**0.93**	0.73	+3.85	**0**
KGW (Kirchenbauer et al., 2023)	$\delta = 2$	16.21	1.40×	1.00	1.00	1.00	+6.93	**0**
KGW (Kirchenbauer et al., 2023)	$\delta = 3$	14.05	1.21×	1.00	1.00	1.00	+8.92	**0**
Post-hoc substitution — ReTrace V1, archived (§4)								
No watermark	$T = 0.8$, full vocab	26.21	1.00×	–	–	–	−0.03	–
ReTrace V1	keyed carrier	27.7	1.11×	0.00	0.00	0.00	+0.73	9
ReTrace V1	entropy gate	41.7	1.59×	0.30	0.20	0.10	+1.49	9
ReTrace V1	leverage, quality-first	30.5	1.17×	0.10	0.10	0.00	+0.58	9
ReTrace V1	leverage, detection-first	65.3	2.49×	0.62	0.50	–	+2.94	9
ReTrace V1	generation-time, two-phase	203.7	7.77×	0.12	0.00	0.00	+1.14	9
Response-guided resampling — ReTrace V3, presence-only, PRELIMINARY (n=20, valid 9–10)								
ReTrace V3	$R = 1$, 256 tok	pending†	–	0.40	0.25	0.25	+1.6*	72
ReTrace V3	$R = 2$, 256 tok	pending†	–	0.60	0.50	0.30	+3.1*	72
ReTrace V3	$R = 1$, 512 tok	pending†	–	0.50	0.40	0.10	+2.2*	144
ReTrace V3	$R = 2$, 512 tok	pending†	–	0.60	0.40	0.20	+2.1*	144

Table 1: Watermark detectability against the text quality actually paid for. All rows measured on LLaDA-8B-Instruct, C4 prompts (300-character truncation), 256 generated tokens, GPT-2-large perplexity; $n = 50$ for dgMARK, $n = 30$ for KGW, $n = 10$ for ReTrace. *Ratio* is perplexity relative to the matching non-watermarked arm directly above it. Each block is compared against its own control because the decoding regime differs: dgMARK uses block decoding with top-k 3, KGW requires strict left-to-right decoding (its green list is seeded by the preceding token, which order-agnostic decoding does not provide), and ReTrace operates on drafts sampled at $T = 0.8$ over the full vocabulary. *Det. fwd.* is the number of model forward passes a detection requires. Non-watermarked arms sit at $|z| \leq 0.25$, so the thresholds are not inflated. KGW is scored on each *distinct* bigram: its green list is seeded by the preceding token, and under forced left-to-right decoding LLaDA repeats roughly a third of its bigrams, so re-scoring repetitions gave $z = +3.32$ and a 37% false positive rate on the non-watermarked $\delta = 0$ arm. Two caveats on KGW's apparently free detectability. Its regime carries a cost the ratio column hides: left-to-right decoding scores 11.61 against 9.94 for block decoding, so measured against the better control $\delta = 1$ costs 1.20×, not 1.03×. And perplexity is not monotone in δ here (16.21 at $\delta = 2$ versus 14.05 at $\delta = 3$), because a strong green-list bias also makes the text more repetitive, which an n-gram-blind perplexity rewards. The V3 rows are preliminary: every carrier feeds one presence test (nulls: 0.498 at n_{sync}=123 for 256 tokens, 0.509 at 148 for 512); detection cost grows linearly with length while the presence count grows with it, which is why R=1 — the only quality-neutral setting — climbs from 0.25 to 0.40 at 1% between the two lengths, the same length-power trade dgMARK reports; *the z implied by the mean match rate; †V3 quality is deliberately left unreported here — all V3 perplexities so far were measured against a *differently-scheduled* reference generator, and R=1 and R=2 showing the same ΔNLL is the signature of that confound rather than of watermark cost. A matched-scheduler control (same generator, steering off) is running; its paired ΔNLL is the number this column will carry. About half of C4-continuation generations are shorter than the evaluable threshold in *both* arms; dgMARK's own protocol filters at 200 tokens for the same reason, and the valid-generation rate will be reported for every method under one shared rule.

Held-out evaluation. The documents used while developing and debugging the method are not a test set. Final numbers are reported on fresh documents with per-document nonces $K_d = \text{HMAC}(K, \text{nonce}_d)$, so each document is an independent key draw and the guarantee of Proposition 1 applies per document.

Nominal α	Mean FPR	Worst-key FPR	Verdict
0.10	0.0121	0.0583	valid ($\leq \alpha$)
0.05	0.0037	0.0250	valid ($\leq \alpha$)
0.01	0.0000	0.0000	valid ($\leq \alpha$)

Table 2: False positive control for a fixed key, measured per key. 20 keys $\times$ 120 non-watermarked LLaDA generations $= 2400$ draws; z over all of them has mean -0.076 and standard deviation 0.976. The decision rule is the one detection actually uses, $p_{\text{bound}} = \exp(-z^2/2)$, Hoeffding's inequality for Rademacher sums. Deployment uses one secret key over many documents, so the relevant guarantee is $P_Y(p \leq \alpha \mid K = k) \leq \alpha$ for the *worst* k, not an average over keys, and a randomization p-value owes super-uniformity rather than strict uniformity. An earlier revision reported a Gaussian tail instead and measured 0.145 at $\alpha = 0.10$ — anti-conservative, and the reason the exactness claim was withdrawn.

6 Results

Preliminary detection (presence-only, 256 tokens, n=20). With every carrier feeding a single presence test, the unwatermarked null sits at a match rate of 0.498 (n_{sync}=123 per document), and the watermarked arms reach TPR@1% $= 0.25$ at R=1 and 0.50 at R=2, doubling with the retry budget exactly as the per-position acceptance analysis predicts. Two things this section still withholds, deliberately. *Quality:* every perplexity so far was measured against a differently-scheduled reference generator, and the watermarked arms at R=1 and R=2 showing the same ΔNLL is the signature of that confound; a matched-scheduler control is running and its paired ΔNLL is the number that will be reported. *Scale:* these are n=20 documents of which about half clear the evaluable-length threshold in both arms — the freeze decision uses $n \geq 30$ valid documents at 512 tokens, where the presence count doubles again.

The mechanism study that led here is summarised by three measurements: the per-position acceptance mass under the live decoder is bimodal (quartiles 0.021/0.601/0.982), a third of carriers hold under 0.1 mass on the side the key requests, and truncating the live row to top-p0.9 leaves a median two-sided mass of 0.000 — inside the model's plausible set the reconstruction response is nearly deterministic, so match rate is bought in the tail of the distribution, which is exactly where text quality is paid.

7 Conclusion

ReTrace separates the two roles a watermark usually conflates. The decoding order is where the watermark is *placed*, cheaply, by choosing when a model-preferred token is committed. The model's reconstruction response is where it is *read back*, from the finished text alone. Whether that separation converts enough steerable positions into detection power at a fixed quality budget is the question the remaining experiments answer.

References

Scott Aaronson and Hendrik Kirchner. Watermarking gpt outputs, 2023. https://www.scottaaronson.com/talks/watermark.ppt.

Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T. Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

Arka Bagchi, Akhil Bhimaraju, Moulik Choraria, Daniel Alabi, and Lav R. Varshney. Watermarking discrete diffusion language models. *arXiv preprint arXiv:2511.02083*, 2025.

Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *NeurIPS*, 2025.

Adam Block, Alexander Rakhlin, and Ayush Sekhari. Gaussmark: A practical approach for structural watermarking of language models. In *ICML*, 2025.

Ruibo Chen, Yihan Wu, Yanshuo Chen, Chenxi Liu, Junfeng Guo, and Heng Huang. A watermark for order-agnostic language models. In *ICLR*, 2025.

Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *COLT*, 2024.

Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. Gumbelsoft: Diversified language model watermarking via the gumbelmax-trick. In *ACL*, 2024.

Thibaud Gloaguen, Robin Staab, Nikola Jovanović, and Martin Vechev. Watermarking diffusion language models. In *ICLR*, 2026. arXiv:2509.24368.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

Pyo Min Hong and Albert No. dgMARK: Decoding-Guided Watermarking for Diffusion Language Models. In *ICML*, 2026. arXiv:2601.22985.

Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *ICML*, 2024.

Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *ICML*, 2025.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *ICML*, 2023.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *ICLR*, 2024.

Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John F. Wieting, and Mohit Iyyer. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. In *NeurIPS*, 2023.

Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *TMLR*, 2024.

Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sarah Birnbaum, Ziyang Luo, Yanis Miraoui, and Akash Palrecha. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

Yuqing Liang, Jiancheng Xiao, Wensheng Gan, and Philip S. Yu. Watermarking techniques for large language models: A survey. *Artificial Intelligence Review*, 2026.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *ICML*, 2024.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *NeurIPS*, 2025.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *ICLR*, 2025.

Fabien A. P. Petitcolas, Ross J. Anderson, and Markus G. Kuhn. Information hiding — a survey. In *Proceedings of the IEEE*, 1999.

Ohad Raban, Ethan Fetaya, and Gal Chechik. LR-DWM: Efficient watermarking for diffusion language models. *arXiv preprint arXiv:2601.12376*, 2026.

Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Marroquin Marroquin, Alexander M. Rush, Yair Schiff, Justin T. Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.

Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Pengfei Zeng, Yu Wang, Biao Qi, Dong Liu, and Linfeng Zhang. Accelerating diffusion large language models with slowfast sampling: The three golden principles. In *ICLR*, 2026.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. In *ICLR*, 2026.

Linyu Wu, Linhao Zhong, Wenjie Qu, Yifan Li, Yue Liu, Shengfang Zhai, Chunhua Shen, and Jiaheng Zhang. DMark: Order-agnostic watermarking for diffusion large language models. *arXiv preprint arXiv:2510.02902*, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Xuandong Zhao, Yu-Xiang Wang, and Lei Li. Protecting language generation models via invisible watermarking. In *ICML*, 2023.

Xuandong Zhao, Prabhanjan V. Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for ai-generated text. In *ICLR*, 2024.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.